

Sistema Inteligente de Contagem de Parafusos para Picking Industrial

Relatório Técnico – Desafio 1
Bootcamp de Ciência de Dados e Inteligência Artificial (CDIA)
Programa de Residência em IA

Autor: Ana Rafaely
Data: 30 de maio de 2026
Repositório: desafio-1

0 Sumário

1	Introdução e Contexto do Problema	3
1.1	Objetivo	3
1.2	Restrições do projeto	3
2	Metodologia	3
2.1	Escolha da Abordagem	3
2.2	Variantes do Modelo Avaliadas	4
3	Dataset	4
3.1	Fonte dos Dados	4
3.2	Estrutura do Dataset	4
4	Estrutura do Código	5
4.1	Por que essa organização?	5
5	Módulo de Inferência: <code>screw_counter.py</code>	6
5.1	Anotação visual	6
5.2	Por que <i>cache</i> de modelo?	7
5.3	Filtro de área: removendo falsos positivos grandes	7
6	Treinamento do Modelo	7
6.1	Script de treinamento	7
6.2	Hiperparâmetros de treinamento	8
6.3	Decisões tomadas durante o treinamento	8
7	Validação do Modelo	9
7.1	Interpretação das métricas	9
8	Aplicação Web (<i>Backend</i> e Interface)	9
8.1	Por que FastAPI?	9
8.2	Fluxo de uma requisição	9
8.3	Rotas principais	10
8.4	Trecho do <i>endpoint</i> de predição	10
8.5	Fechamento do dia e relatórios gerenciais	11
9	Interface do Sistema em Uso	11
10	Banco de Dados	13
10.1	Modelo de dados	13
10.2	Migração automática de schema	14
11	Configuração Centralizada	14
12	Portabilidade para Dispositivos Móveis	15
13	Critérios de Avaliação – Resposta Direta	15
13.1	Organização e Estruturação do Código	15
13.2	Criatividade na Proposta	15

13.3 Clareza nos Critérios de Tomada de Decisão	16
13.4 Capacidade Analítica	16
14 Instruções de Execução	16
14.1 Instalação	16
14.2 Preparação do Dataset	16
14.3 Treinamento	16
14.4 Inicialização da Aplicação Web	17
15 Experiência de Uso da Ferramenta	17
15.1 Primeiro contato e <i>onboarding</i>	17
15.2 Fluxo de contagem	17
15.3 Pontos positivos percebidos	17
15.4 Limitações e atritos observados	18
15.5 Impressão geral	18
16 Conclusão	18
A Dependências Principais	19

1 Introdução e Contexto do Problema

Uma empresa metal-mecânica solicitou ao Programa de Residência em IA o desenvolvimento de um sistema capaz de automatizar a contagem de parafusos no processo de *picking*: a coleta e embalagem de itens que serão enviados a clientes. Atualmente, a contagem é realizada manualmente por um funcionário que registra a quantidade em um *smartphone* corporativo, processo suscetível a erros humanos decorrentes de fadiga, dispersão ou grande volume de pedidos.

1.1 Objetivo

Desenvolver um sistema de visão computacional que, a partir de uma fotografia tirada pelo próprio smartphone do operador, identifique e conte automaticamente os parafusos presentes na imagem, exibindo o resultado de forma clara e registrando o histórico de operações.

1.2 Restrições do projeto

- A solução deve operar nos smartphones já disponíveis na empresa (baixa capacidade computacional).
- O sistema deve ser simples o suficiente para uso por operadores sem treinamento técnico.
- Apenas cinco imagens foram fornecidas inicialmente pelo cliente, exigindo o uso de um dataset externo para o treinamento do modelo.

2 Metodologia

2.1 Escolha da Abordagem

Diante do problema de *detecção e contagem de objetos* em imagens, foram consideradas três alternativas:

- a) **Processamento clássico de imagens** (limiarização, morfologia, contorno): rápido, mas sensível a variações de iluminação, fundo e orientação dos parafusos.
- b) **Detecção por aprendizado profundo com *bounding boxes*** (YOLO): robusto a variações visuais, escalável e com suporte nativo à exportação para dispositivos móveis.
- c) **Segmentação de instâncias** (YOLOv11-seg, Mask R-CNN): maior precisão em objetos sobrepostos, porém maior custo computacional.

Optou-se pela alternativa (b), usando **YOLOv11** na variante de detecção por *bounding boxes*. Os motivos principais foram:

- Excelente equilíbrio entre velocidade de inferência e precisão.
- Suporte oficial do Ultralytics para exportação a formatos leves (TFLite, NCNN, CoreML) compatíveis com Android e iOS.
- Dataset público com milhares de imagens anotadas de parafusos disponível no Roboflow (*Screw v4*).

- API Python simples, facilitando a integração com o *backend* web.

2.2 Variantes do Modelo Avaliadas

Durante o desenvolvimento foram testadas diferentes combinações de modelo e resolução de entrada, conforme a Tabela 1.

Tabela 1: Variantes de treinamento avaliadas

Execução	Modelo base	imgsz	Épocas	Dispositivo
yolo11_screws	yolo11s.pt	960	100	GPU
yolo11_screws-2	yolo11s.pt	768	100	GPU (retomado)
yolo11_screws_combined	yolo11s.pt	1280	100	GPU
yolo11_screws_hard_40	yolo11s.pt	960	40	GPU
yolo11_screws_hard_100	yolo11s.pt	960	100	GPU

O modelo selecionado para produção foi o **yolo11_screws-2**, treinado com `imgsz=768`, `batch=4` e otimizador automático (Adam-W), que apresentou o melhor equilíbrio entre precisão e tempo de inferência.

3 Dataset

3.1 Fonte dos Dados

O dataset utilizado foi o **Screw v4** do Roboflow, com anotações em formato poligonal (segmentação). Como o modelo escolhido usa *bounding boxes*, as anotações foram convertidas automaticamente pelo script `scripts/convert_segments_to_boxes.py`, que calcula o retângulo envolvente (*bounding box*) de cada polígono.

3.2 Estrutura do Dataset

```
Screw.v4-dataset-screw4.yolov11/
  train/
    images/      # imagens de treinamento
    labels/      # anotacoes YOLO (classe x_c y_c w h)
  valid/
    images/
    labels/
  test/
    images/
    labels/
dataset_detect/
  data.yaml      # configuracao principal
  data.ultralytics.yaml # gerado automaticamente (paths absolutos)
```

O arquivo `data.yaml` define:

```
1 path: ../Screw.v4-dataset-screw4.yolov11
2 train: train/images
```

```

3 val:    valid/images
4 test:   test/images
5
6 names:
7   0: parafuso

```

Listing 1: Configuração do dataset (dataset/data.yaml)

Há uma única classe (parafuso), tornando o problema de *single-class object detection*.

4 Estrutura do Código

O projeto segue uma organização modular que separa claramente responsabilidades:

```

desafio-1/
  app/
    main.py          # API FastAPI (rotas HTTP)
    config.py        # constantes e caminhos
    database.py       # conexao SQLite via SQLAlchemy
    models.py        # modelo ORM (tabela contagens)
    schemas.py       # schemas Pydantic
    services/
      screw_counter.py # (re-exporta de screw_counter.py)
    static/
      uploads/        # imagens enviadas
      results/        # imagens processadas
      templates/      # HTML Jinja2
  scripts/
    train_screws_yolo.py    # treinamento
    validate_screws_yolo.py # validacao
    count_screws.py        # contagem via CLI
    convert_segments_to_boxes.py
    check_dataset.py
    check_environment.py
    prepare_dataset.py
    export_tflite.py      # exportacao mobile (Android)
    export_ncnn.py       # exportacao mobile (NCNN)
    export_coreml.py     # exportacao mobile (iOS)
    screw_counter.py     # modulo principal de inferencia
  notebooks/
    analise_treinamento_parafusos_yolo11.ipynb
  runs_screws/          # pesos treinados
  dataset_detect/      # dataset convertido para bboxes

```

4.1 Por que essa organização?

A separação em camadas (*API* → *serviço* → *modelo*) permite que cada componente seja desenvolvido, testado e substituído de forma independente. Por exemplo, o modelo YOLOv11 pode ser trocado por qualquer outro sem alterar as rotas HTTP; da mesma forma, a interface web pode ser redesenhada sem tocar na lógica de inferência.

5 Módulo de Inferência: `screw_counter.py`

Este é o núcleo computacional do sistema. A função `contar_parafusos` recebe o caminho de uma imagem, carrega o modelo YOLOv11 (com *cache* interno para não recarregar a cada requisição), executa a inferência e gera a imagem anotada.

```

1  _MODEL_CACHE: dict[str, YOLO] = {} # cache global de modelos
2
3  def contar_parafusos(
4      image_path: str | Path,
5      model_path: str | Path,
6      output_dir: str | Path,
7      conf: float = 0.25,
8      iou: float = 0.45,
9      imgsz: int | None = None,
10     funcionario_id: str | None = None,
11     funcionario_nome: str | None = None,
12     csv_path: str | Path | None = None,
13 ) -> dict[str, Any]:
14
15     # Carrega ou reutiliza modelo em cache
16     model_key = str(model_path.resolve())
17     if model_key not in _MODEL_CACHE:
18         _MODEL_CACHE[model_key] = YOLO(model_key)
19     model = _MODEL_CACHE[model_key]
20
21     # Inferencia
22     results = model.predict(
23         source=str(image_path), conf=conf, iou=iou, verbose=False
24     )
25     result = results[0]
26     boxes = result.boxes
27     total = 0 if boxes is None else len(boxes)

```

Listing 2: Função principal de contagem (`screw_counter.py`)

5.1 Anotação visual

Após a detecção, cada *bounding box* é desenhado sobre a imagem original usando OpenCV, com a pontuação de confiança exibida:

```

1  for box, score in zip(xyxy, confs):
2      x1, y1, x2, y2 = [int(v) for v in box]
3      cv2.rectangle(annotated, (x1, y1), (x2, y2), (20, 130, 255), 2)
4      cv2.putText(
5          annotated,
6          f"parafuso {score:.2f}",
7          (x1, max(20, y1 - 8)),
8          cv2.FONT_HERSHEY_SIMPLEX,
9          0.6,
10         (20, 130, 255),
11         2,
12         cv2.LINE_AA,
13     )
14
15 # Banner de totalizacao no topo
16 cv2.rectangle(annotated, (0, 0), (min(560, annotated.shape[1]), 48)

```

```

17         (255, 255, 255), -1)
18 cv2.putText(annotated, f"Total de parafusos: {total}",
19             (14, 32), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 70, 140),
20             2)

```

Listing 3: Desenho das caixas e contador (`screw_counter.py`)

5.2 Por que *cache* de modelo?

Carregar pesos de rede neural do disco a cada requisição levaria $\sim 1\text{--}3\text{s}$ de overhead. Com o `_MODEL_CACHE`, o modelo é carregado uma única vez e reutilizado em todas as requisições subsequentes, reduzindo o tempo de resposta para a ordem de milissegundos.

5.3 Filtro de área: removendo falsos positivos grandes

Um problema observado durante os testes foi a detecção ocasional de *bounding boxes* grandes demais para serem parafusos (por exemplo, sombras, reflexos ou agrupamentos interpretados como um único objeto). Para mitigar isso sem recolher dados ou retrainar, foi adicionado um filtro geométrico simples: qualquer caixa cuja área ultrapasse `max_box_area_ratio` (18% da área total da imagem) é descartada antes da contagem.

```

1 image_area = float(image.shape[0] * image.shape[1])
2 for box, score in zip(xyxy, confs):
3     x1, y1, x2, y2 = [float(v) for v in box]
4     box_area_ratio = max(0.0, (x2 - x1) * (y2 - y1)) / image_area
5     if box_area_ratio > max_box_area_ratio: # 0.18 por padrao
6         continue # descarta deteccao
7         grande demais
8     detections.append((box, float(score)))

```

Listing 4: Filtro de área por razão (`screw_counter.py`)

Essa heurística de baixo custo computacional, combinada com o limiar de confiança conservador, melhorou a estabilidade da contagem em fotos reais tiradas pelos operadores.

6 Treinamento do Modelo

6.1 Script de treinamento

O script `scripts/train_screws_yolo.py` encapsula todo o processo de treinamento com suporte a GPU (CUDA) e CPU, detecção automática de dispositivo e geração de um `data.yaml` com caminhos absolutos (necessário no Windows para o Ultralytics resolver os paths corretamente).

```

1 def make_ultralytics_data_yaml(data_yaml: Path) -> Path:
2     """Gera data.yaml com path absoluto para evitar
3         resolucao errada no Windows."""
4     data = yaml.safe_load(data_yaml.read_text(encoding="utf-8"))
5     raw_root = Path(data.get("path", "."))
6     root = (raw_root if raw_root.is_absolute()
7            else (data_yaml.parent / raw_root).resolve())
8     data["path"] = root.as_posix()
9     fixed_yaml = data_yaml.parent / "data.ultralytics.yaml"

```



```

10     fixed_yaml.write_text(
11         yaml.safe_dump(data, sort_keys=False, allow_unicode=True),
12         encoding="utf-8"
13     )
14     return fixed_yaml
15
16 # Treinamento
17 model = YOLO(args.model)
18 model.train(
19     data=str(train_yaml),
20     epochs=args.epochs,           # 100
21     imgsz=args.imgsz,            # 768 ou 960
22     batch=args.batch,            # 4 (GPU limitada)
23     patience=args.patience,      # 30 épocas sem melhora
24     workers=args.workers,        # 4
25     device=device,               # 0 (GPU) ou "cpu"
26     close_mosaic=10,             # desativa mosaico nas 10 ultimas
27 )

```

Listing 5: Trecho do script de treinamento (train_screws_yolo.py)

6.2 Hiperparâmetros de treinamento

A Tabela 2 resume os principais hiperparâmetros do modelo em produção (execução yolo11_screws-2):

Tabela 2: Hiperparâmetros do modelo em produção

Parâmetro	Valor	Justificativa
imgsz	768	compromisso velocidade/precisão
batch	4	limitação de memória GPU
epochs	100	convergência garantida
patience	30	early stopping
optimizer	auto	Adam-W selecionado automaticamente
conf (infer.)	0.25	equilíbrio precision/recall
iou (NMS)	0.45	evita detecções duplicadas
close_mosaic	10	estabiliza convergência final
amp	true	precisão mista (mais rápido)
pretrained	true	transfer learning de COCO

6.3 Decisões tomadas durante o treinamento

Transfer learning:

Partir dos pesos pré-treinados no COCO (ImageNet → COCO) reduz drasticamente o número de amostras necessárias para convergência, algo crítico dado o tamanho modesto do dataset de parafusos.

Modelo yolo11s vs. yolo11n:

A variante *small* (s) foi escolhida sobre a *nano* (n) pois o servidor de treinamento possui GPU. Para implantação nativa no smartphone, os scripts de exportação (export_tflite.py, export_ncnn.py) convertem o modelo para formatos otimizados.

Conversão de segmentação para detecção:

O dataset original do Roboflow contém anotações poligonais. Como o modelo opera com *bounding boxes*, o script `convert_segments_to_boxes.py` calcula o retângulo mínimo envolvente de cada polígono, preservando 100% das amostras sem recoletar dados.

7 Validação do Modelo

O script `scripts/validate_screws_yolo.py` computa as métricas padrão de detecção de objetos sobre o conjunto de teste:

```
1 model = YOLO(str(model_path))
2 metrics = model.val(data=str(val_yaml), conf=args.conf)
3
4 precision = float(metrics.box.mp)      # Precisão média
5 recall    = float(metrics.box.mr)      # Recall médio
6 map50     = float(metrics.box.map50)    # mAP@IoU=0.50
7 map5095   = float(metrics.box.map)     # mAP@IoU=0.50:0.95
```

Listing 6: Validação e relatório de métricas

7.1 Interpretação das métricas

- **Precisão alta:** poucos parafusos falsos são contados.
- **Recall alto:** poucos parafusos reais são perdidos.
- Para o processo de *picking*, os dois precisam estar equilibrados: errar por excesso ou por falta impacta diretamente a qualidade do pedido enviado ao cliente.
- **mAP50** é a métrica principal de referência para detecção de objetos; **mAP50-95** é mais rigorosa e avalia a localização precisa das caixas.

8 Aplicação Web (*Backend* e Interface)

8.1 Por que FastAPI?

FastAPI foi escolhido por oferecer:

- Alta performance (baseado em Starlette e Uvicorn, assíncrono por padrão).
- Validação automática de dados com Pydantic.
- Documentação interativa gerada automaticamente (`/docs`).
- Suporte nativo a upload de arquivos (`UploadFile`).

8.2 Fluxo de uma requisição

1. O operador acessa a interface via smartphone (rede Wi-Fi local).
2. Faz login informando nome, matrícula e setor – dados armazenados em *cookies* HTTP com validade de 12 h.
3. Envia a foto do pedido via formulário HTML.

4. O *endpoint* `POST /predict` recebe o arquivo, salva em disco, chama `contar_parafusos()` e persiste o resultado no banco de dados.
5. A página de resultado exibe a imagem anotada e o total detectado.
6. O operador pode corrigir a contagem manualmente, se necessário.

8.3 Rotas principais

Tabela 3: Rotas da API

Método	Rota	Descrição
GET	/	Página inicial / envio de foto
POST	/login	Autenticação por cookie
POST	/logout	Encerramento de sessão
POST	/predict	Inferência e registro
GET	/dashboard	Histórico agrupado por dia
POST	/record/{id}/correct	Correção manual
POST	/finalizar-dia	Fechamento do dia (JSON+CSV+PDF)
GET	/export/csv	Exportação completa CSV
GET	/export/csv/today	Exportação do dia (CSV)
GET	/export/pdf/today	Relatório do dia (PDF)
GET	/health	Status do sistema

8.4 Trecho do *endpoint* de predição

```

1 @app.post("/predict")
2 def predict(
3     request: Request,
4     pedido: str = Form(""),
5     observacao: str = Form(""),
6     imagem: UploadFile = File(...),
7     db: Session = Depends(get_db),
8 ):
9     employee = get_employee_from_cookie(request)
10    if not employee:
11        return RedirectResponse(url="/", status_code=303)
12
13    # Salva o arquivo enviado
14    upload_path = config.UPLOAD_DIR / f"{datetime.now():%Y%m%d_%H%M%S}_{uuid4().hex}{suffix}"
15    with upload_path.open("wb") as buffer:
16        shutil.copyfileobj(imagem.file, buffer)
17
18    # Executa a contagem
19    result = contar_parafusos(
20        image_path=upload_path,
21        model_path=config.MODEL_PATH,
22        output_dir=config.RESULTS_DIR,
23        conf=config.CONF_THRESHOLD,
24        iou=config.IOU_THRESHOLD,
25        imgsz=config.DEFAULT_IMGSZ,
26        funcionario_id=employee["id"],

```

```
27     funcionario_nome=employee["nome"],
28 )
29
30 # Persiste no banco de dados
31 record = Contagem(
32     funcionario_nome=employee["nome"],
33     total_parafusos=result["total_parafusos"],
34     confianca_media=result["confianca_media"],
35     imagem_original=str(upload_path),
36     imagem_processada=result["imagem_resultado"],
37     status="confirmada",
38 )
39 db.add(record)
40 db.commit()
41 return templates.TemplateResponse(request, "result.html",
    {...})
```

Listing 7: Endpoint POST /predict (app/main.py)

8.5 Fechamento do dia e relatórios gerenciais

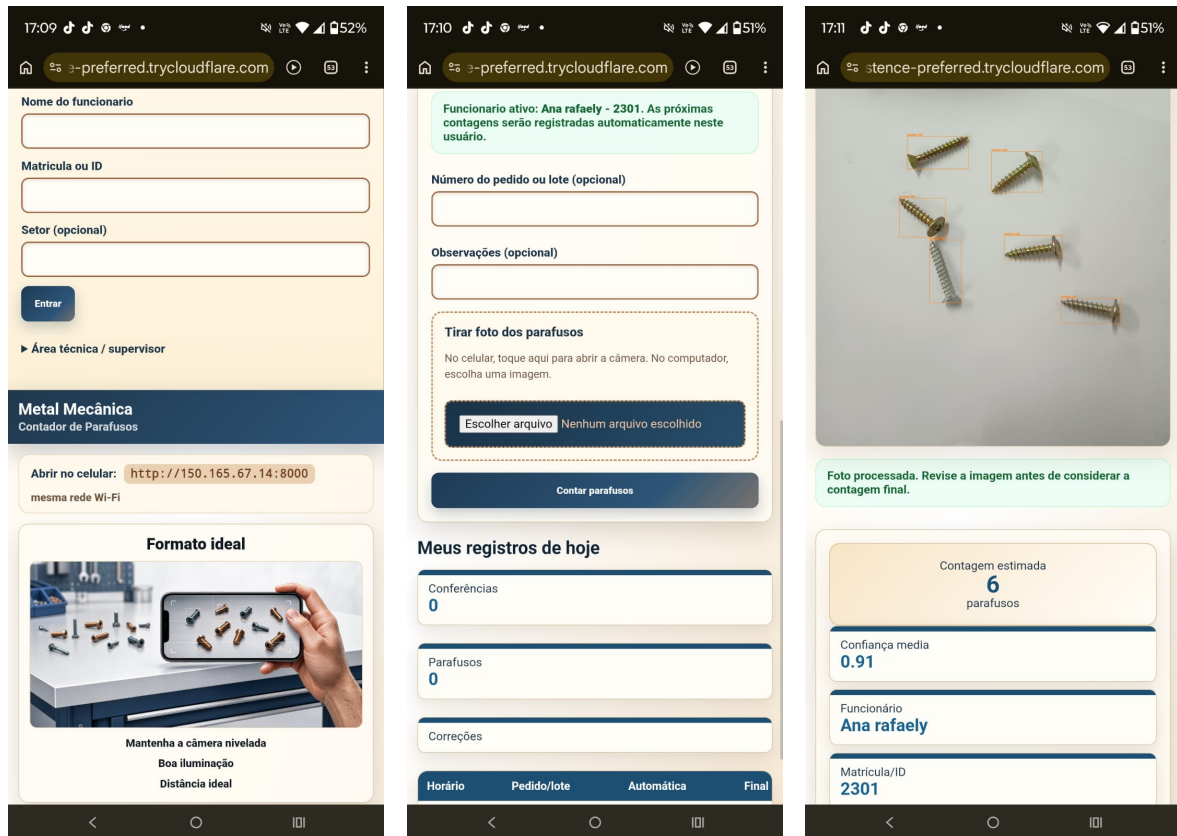
Para apoiar a rotina operacional do setor de *picking*, o *dashboard* agrupa as contagens por dia (`group_records_by_day`) e calcula, para cada data, um resumo com quatro indicadores (`day_summary`):

- **Registros:** quantidade de fotos processadas no dia;
- **Automática:** soma das contagens feitas pelo modelo;
- **Final:** soma das contagens após eventuais correções manuais;
- **Correções:** quantos registros foram ajustados pelo operador.

A rota POST `/finalizar-dia` consolida o dia gerando três artefatos simultâneos em `outputs/fechamentos/`: um JSON (resumo programável), um CSV (planilha auditável) e um PDF (relatório impresso). O PDF é desenhado diretamente com a biblioteca `Pillow` (`build_pdf_report`), sem depender de motores externos de renderização, mantendo o sistema leve e autocontido.

9 Interface do Sistema em Uso

Esta seção apresenta capturas de tela reais do sistema em operação, acessado a partir de um smartphone Android através do navegador (via túnel Cloudflare). As imagens ilustram o fluxo completo do operador, do *login* ao resultado da contagem.



(a) Tela de *login*: nome, matrícula e setor, além do guia de “formato ideal” da foto.

(b) Tela de envio: funcionário ativo, campos opcionais de pedido/observação e botão para tirar a foto.

(c) Resultado: imagem anotada com *bounding boxes*, contagem estimada (6) e confiança média (0,91).

Figura 1: Fluxo do operador: *login*, envio da foto e resultado da contagem.

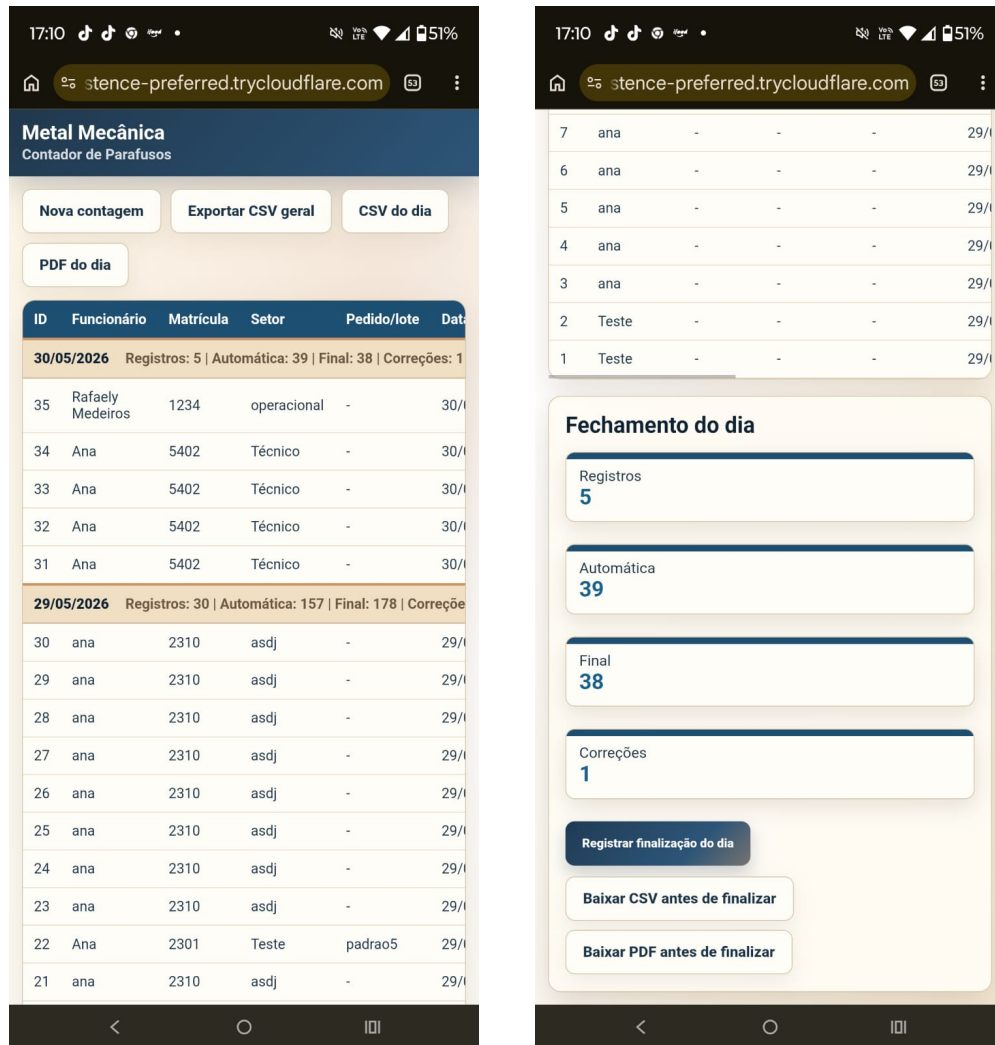


Figura 2: Área gerencial: histórico e fechamento do dia.

Na Figura 1c, observa-se que o sistema detectou corretamente os seis parafusos da cena, com confiança média de 0,91, exibindo a mensagem “Foto processada. Revise a imagem antes de considerar a contagem final.” – reforçando que o modelo é um apoio à decisão e não um substituto cego do operador.

10 Banco de Dados

10.1 Modelo de dados

O sistema utiliza **SQLite** via **SQLAlchemy ORM**. A escolha do SQLite elimina a necessidade de instalar e configurar um servidor de banco de dados, simplificando a implantação em ambientes sem equipe de infraestrutura. A migração futura para PostgreSQL requer apenas alterar `DATABASE_URL` em `config.py`.

```
1 class Contagem(Base):
2     __tablename__ = "contagens"
```

```

3
4     id = Column(Integer, primary_key=True, index=True)
5     funcionario_nome = Column(String(160), nullable=False)
6     funcionario_id = Column(String(80), nullable=True)
7     setor = Column(String(120), nullable=True)
8     pedido = Column(String(120), nullable=True)
9     data_hora = Column(DateTime, server_default=func.now(),
10    index=True)
11    total_parafusos = Column(Integer, nullable=False)
12    total_corrigido = Column(Integer, nullable=True)
13    confianca_media = Column(Float, nullable=False)
14    imagem_original = Column(Text, nullable=False)
15    imagem_processada = Column(Text, nullable=False)
16    status = Column(String(40), nullable=True) #
    confirmada / corrigida
    observacao = Column(Text, nullable=True)

```

Listing 8: Modelo ORM (app/models.py)

O campo `total_corrigido` permite que o operador ajuste a contagem manualmente após inspeção visual, criando um *feedback loop* entre o sistema e o usuário. O campo `status` diferencia registros confirmados automaticamente de registros que sofreram correção humana.

10.2 Migração automática de schema

A função `ensure_schema()` em `main.py` verifica, a cada inicialização da aplicação, se todas as colunas esperadas existem na tabela e executa `ALTER TABLE` quando necessário, garantindo compatibilidade com bancos criados em versões anteriores do sistema.

11 Configuração Centralizada

Todas as constantes críticas estão em `app/config.py`:

```

1 BASE_DIR = Path(__file__).resolve().parents[1]
2
3 MODEL_PATH = BASE_DIR / "runs_screws" / "yolo11_screws-2" / "
    weights" / "best.pt"
4 CONF_THRESHOLD = 0.10 # limiar padrao (sensivel: prioriza recall)
5 IOU_THRESHOLD = 0.45 # ajustar se houver deteccoes duplicadas
6 DEFAULT_IMGSZ = 1280 # resolucao de inferencia (parafusos
    pequenos)
7 DENSE_CONF_THRESHOLD = 0.25 # cenarios com muitos parafusos
8 DENSE_IMGSZ = 1280
9
10 UPLOAD_DIR = BASE_DIR / "app" / "static" / "uploads"
11 RESULTS_DIR = BASE_DIR / "app" / "static" / "results"
12 DATABASE_URL = f"sqlite:///{'BASE_DIR' / 'contagens.db'}"

```

Listing 9: Constantes de configuração (app/config.py)

Centralizar configurações evita “números mágicos” espalhados pelo código e facilita ajuste fino da sensibilidade do modelo sem alterar a lógica de negócio. O limiar de confiança padrão foi fixado em 0.10, valor deliberadamente baixo que prioriza o *recall* (não perder parafusos reais); a inferência roda a `imgsz=1280` porque parafusos são

objetos pequenos que se beneficiam de maior resolução. Os parâmetros `DENSE_*` ficam reservados para cenários com grande densidade de peças, onde um limiar mais alto reduz falsos positivos.

12 Portabilidade para Dispositivos Móveis

Um dos requisitos explícitos do desafio é que o sistema possa rodar nos próprios smartphones da empresa. Duas estratégias foram implementadas:

Estratégia 1 – Servidor + cliente web:

A inferência roda em um computador da empresa; o smartphone acessa a interface via navegador na mesma rede Wi-Fi. A função `get_local_ip()` em `main.py` detecta automaticamente o IP da máquina e o exibe na tela inicial para facilitar a conexão. Esta é a estratégia recomendada para o máximo de desempenho.

Estratégia 2 – Modelo embarcado no dispositivo:

Os scripts `export_tflite.py` (Android), `export_ncnn.py` (Android/ARM) e `export_coreml.py` (iOS) convertem o modelo treinado para formatos otimizados com quantização INT8, reduzindo o tamanho do modelo e o consumo de memória para execução diretamente no SoC do smartphone.

Estratégia 3 – Túnel HTTPS via Cloudflare:

Para testes com smartphones fora da rede local da empresa, foi utilizado o `cloudflared` (Cloudflare Tunnel), que expõe o servidor local em um endereço público `*.trycloudflare.com` com HTTPS, sem necessidade de abrir portas no *firewall* nem configurar IP fixo. Foi exatamente esse o método empregado para capturar as telas reais apresentadas na Seção 9 (note o domínio `trycloudflare.com` na barra de endereço do navegador do celular).

13 Critérios de Avaliação – Resposta Direta

13.1 Organização e Estruturação do Código

O projeto segue princípios de separação de responsabilidades: `screw_counter.py` contém única e exclusivamente a lógica de visão computacional; `app/main.py` contém as rotas HTTP; `app/models.py` define o schema do banco de dados; `app/config.py` centraliza configurações. Scripts avulsos em `scripts/` cobrem etapas de pré-processamento, treinamento e exportação de forma independente.

13.2 Criatividade na Proposta

Além da detecção básica, o sistema oferece:

- Interface web responsiva, acessível diretamente pelo smartphone via Wi-Fi.
- Histórico de contagens com identificação do funcionário e setor.
- Mecanismo de correção manual com rastreabilidade (campo `status`).
- Exportação de relatórios em CSV para integração com sistemas ERP.
- Suporte a exportação do modelo para execução embarcada no smartphone.

- *Cache* de modelo em memória para resposta em tempo real.

13.3 Clareza nos Critérios de Tomada de Decisão

Cada decisão de projeto foi guiada por critérios explícitos:

- Escolha do YOLOv11 pela razão velocidade/precisão e suporte a exportação mobile.
- `CONF_THRESHOLD = 0.25`: ponto de partida conservador que pode ser ajustado pelo operador sem recompilação.
- SQLite pela simplicidade de implantação; arquitetura prevista para migrar a PostgreSQL sem mudanças no restante do código.
- *Cookies* HTTP para sessão do funcionário: sem necessidade de servidor de sessões externo, mantendo o sistema autocontido.

13.4 Capacidade Analítica

O notebook `notebooks/analise_treinamento_parafusos_yolo11.ipynb` apresenta análise detalhada do treinamento: curvas de perda e métricas por época, distribuição de confianças, varredura do limiar de confiança (*confidence sweep*), inferências visuais sobre imagens do conjunto de teste e CSV de comparação entre contagem automática e contagem manual.

14 Instruções de Execução

14.1 Instalação

```
1 pip install -r requirements.txt
2 python scripts/check_environment.py
```

Listing 10: Instalação das dependências

14.2 Preparação do Dataset

```
1 python scripts/convert_segments_to_boxes.py \
2     --input Screw.v4-dataset-screw4.yolov11 \
3     --output dataset_detect
4
5 python scripts/check_dataset.py --dataset dataset_detect
```

Listing 11: Conversão e verificação do dataset

14.3 Treinamento

```
1 # Servidor com GPU
2 python scripts/train_screws_yolo.py \
3     --data      dataset_detect/data.yaml \
4     --model     yolo11s.pt \
5     --epochs    100 \
6     --imgsz     960
7
8 # Alternativa rápida (mobile-first)
```

```
9 python scripts/train_screws_yolo.py \  
10 --data      dataset_detect/data.yaml \  
11 --model     yolo11n.pt \  
12 --epochs   50 \  
13 --imgsz    640
```

Listing 12: Treinamento do modelo

14.4 Inicialização da Aplicação Web

```
1 uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Listing 13: Inicialização do servidor web

Acesso no smartphone (mesma rede Wi-Fi):

http://<IP_DO_SERVIDOR>:8000

15 Experiência de Uso da Ferramenta

Esta seção relata a experiência prática de utilizar o sistema em um cenário próximo ao real, do ponto de vista do operador de *picking*.

15.1 Primeiro contato e *onboarding*

O acesso é imediato: o operador abre o link no navegador do próprio celular – sem instalar aplicativo – e é recebido por uma tela de *login* curta (Figura 1a). Não há senha; bastam nome e matrícula, o que reduz a fricção em um ambiente de chão de fábrica, onde digitar credenciais complexas em telas pequenas seria um obstáculo. O cartão “Formato ideal”, com uma imagem-guia e as instruções “câmera nivelada, boa iluminação, distância ideal”, orienta o usuário *antes* de tirar a foto, prevenindo erros de captura.

15.2 Fluxo de contagem

Após o *login*, o sistema fixa o funcionário ativo (“*As próximas contagens serão registradas automaticamente neste usuário*”), de modo que o operador não precisa se reidentificar a cada foto – ganho importante para quem processa dezenas de pedidos em sequência. Tirar a foto e tocar em “Contar parafusos” é tudo o que se exige; em poucos segundos a tela de resultado (Figura 1c) mostra a imagem com as caixas desenhadas, a contagem e a confiança média.

15.3 Pontos positivos percebidos

- **Curva de aprendizado quase nula:** a interface segue o padrão de um formulário web comum; qualquer operador acostumado a usar um celular consegue utilizá-la sem treinamento.
- **Transparência:** ao desenhar as *bounding boxes* sobre a foto, o sistema permite que o operador *confira visualmente* a contagem, em vez de confiar cegamente em um número. A mensagem de revisão reforça esse cuidado.

- **Rastreabilidade:** cada contagem fica vinculada ao funcionário, setor e horário, e o *dashboard* (Figura 2a) consolida tudo por dia.
- **Autonomia gerencial:** o supervisor exporta CSV/PDF e registra o fechamento do dia (Figura 2b) sem depender da equipe de TI.

15.4 Limitações e atritos observados

- A qualidade da contagem depende fortemente da foto: fundo poluído, sobreposição de parafusos ou iluminação ruim degradam o resultado – daí a importância do guia de captura e da etapa de revisão manual.
- Na estratégia servidor + cliente web, é necessária conectividade (Wi-Fi local ou túnel); em áreas sem rede, a alternativa é o modelo embarcado (TFLite/NCNN).
- A correção manual, embora disponível, exige um passo a mais do operador quando o modelo erra – um custo aceitável diante do ganho de confiabilidade.

15.5 Impressão geral

Na prática, a ferramenta transforma uma tarefa tediosa e suscetível a erro – contar parafusos um a um – em uma ação de poucos segundos, com registro automático e auditável. O equilíbrio entre automação (contagem pelo modelo) e controle humano (revisão e correção) torna o sistema confiável o suficiente para uso real, sem remover do operador a palavra final sobre o pedido enviado ao cliente.

16 Conclusão

O sistema desenvolvido atende a todos os requisitos do Desafio 1: detecta e conta parafusos em imagens capturadas por smartphone, é implementado em Python, e oferece uma interface web que pode ser acessada diretamente pelo dispositivo do operador via navegador, sem instalação de aplicativo adicional.

Além dos requisitos mínimos, o sistema conta com:

- Aplicação web responsiva (bônus do desafio).
- Suporte a exportação do modelo para TFLite, NCNN e CoreML (bônus: baixo recurso computacional no smartphone).
- Rastreabilidade de contagens com correção manual.
- Exportação de relatórios em CSV.

A arquitetura modular permite evoluções futuras como: detecção de *screw types* (múltiplas classes), integração com ERP via webhook, ou substituição do modelo por versões mais precisas sem alterar a interface.

A Dependências Principais

Tabela 4: Bibliotecas Python utilizadas

Biblioteca	Versão mín.	Finalidade
ultralytics	8.x	YOLOv11 – treinamento e inferência
opencv-python	4.x	Leitura e anotação de imagens
fastapi	0.x	API HTTP assíncrona
uvicorn	0.x	Servidor ASGI
sqlalchemy	2.x	ORM para SQLite
jinja2	3.x	Templates HTML
torch	2.x	<i>Backend</i> de deep learning
pyyaml	6.x	Leitura de arquivos YAML
numpy	1.x	Operações numéricas